GENERATIVE AI INTERNSHIP

MONTH-1 PROJECT REPORT

Domain: Generative AI

Tasks Completed

Task 1 :
Build a Streamlit Interface for Inference of a Locally Hosted LLM

Task 2:
Medical Fine-Tuning with Quantization and Low Rank Adaptation via Unsloth in Google Colab

Submitted By :

Name: GANDE SANNIHITHA

Internship Domain: Generative AI

Organization: Arch Technologies

Email: sannihitha.gande04@gmail.com

Phone: 9347157778

Internship Month

Month 1

Academic Year

2026

INDEX

S. No.	Contents	Page No.
1	Task 1 – Streamlit Interface for Inference of a Locally Hosted LLM	2
1.1	Objective	2
1.2	Technologies Used	2
1.3	Ollama and Local LLM Setup	2
1.4	Streamlit Application Development	3
1.5	Conversation History and Chat Management	3
1.6	File and Image Upload	3
1.7	Task 1 Workflow and Result	3
2	Task 2 – Medical Fine-Tuning with Quantization and LoRA using Unsloth	4
2.1	Objective	4
2.2	Google Colab and T4 GPU Setup	4
2.3	Base Model and Medical Dataset	4
2.4	Dataset Preparation	5
2.5	Quantization, LoRA and QLoRA	5
2.6	Training and Monitoring	5
2.7	Loss Comparison	6
2.8	Saving and Testing the Adapter	6
3	Integration of Task 1 and Task 2	7
4	Learning Outcomes and Conclusion	7

TASK 1

STREAMLIT INTERFACE FOR INFERENCE OF A LOCALLY HOSTED LLM

1.1 Objective

The objective of Task 1 was to build an interactive **Streamlit interface for inference of a locally hosted Large Language Model (LLM)**. The application was designed to accept user text input, send the input to a locally running LLM through Ollama, display the generated response, maintain conversation history, and provide a reset/new-chat functionality.

The application was developed as a local chatbot named **Nexa Chatbot**.

1.2 Technologies Used

The following technologies were used for Task 1:

- **Python** – application development
- **Streamlit** – web-based user interface
- **Ollama** – local LLM runtime
- **Llama 3.2** – locally hosted language model
- **JSON** – persistent chat-history storage
- **Windows** – local development environment

1.3 Ollama and Local LLM Setup

The first step was to install Ollama on the local Windows system. The project directory was accessed through the Windows terminal.The project files were checked using commands such as:

```
cd D:\Downloads
cd .\llm-examples-main\llm-examples-main\
dir
```

Ollama was then verified and the Llama 3.2 model was downloaded:

```
ollama --version
ollama pull llama3.2
ollama list
```

The ollama list command was used to verify that the model was available locally. The Streamlit application was then connected to this locally hosted model.

This approach allowed the chatbot to perform inference locally instead of depending on a remote LLM API.

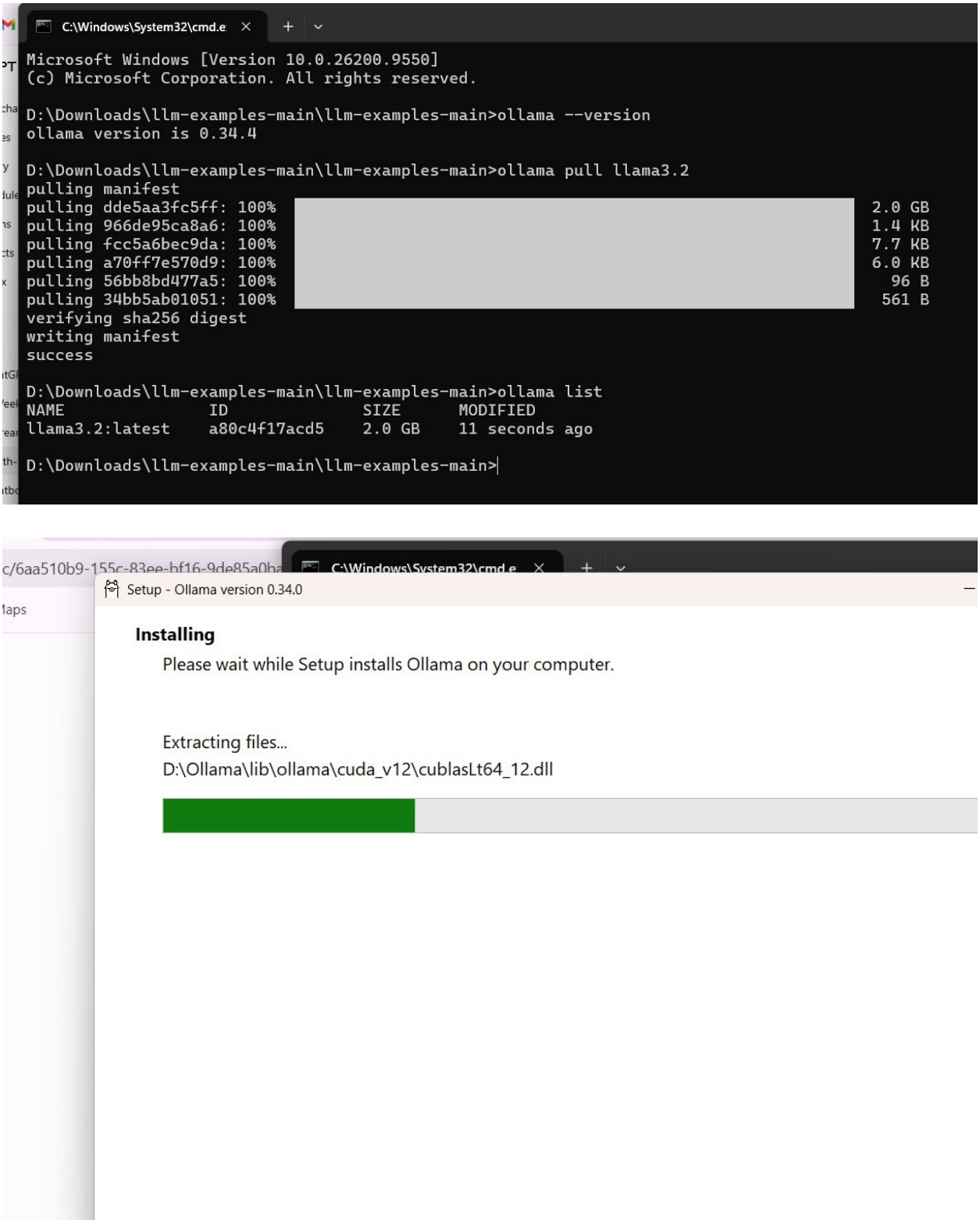


Figure 1: Ollama installation and Llama 3.2 local model setup

TASK 1 IMPLEMENTATION

1.4 Streamlit Application Development

After setting up Ollama, the Python project was connected to the local LLM. Streamlit was used to create the chatbot interface.

The application accepts a user's message and sends the conversation to Ollama using the Python Ollama library.

The main inference operation follows the structure:

```
response = ollama.chat(
    model=MODEL_NAME,
    messages=st.session_state.messages
)

assistant_message = response["message"]["content"]
```

The model used by the application was:

```
MODEL_NAME = "llama3.2"
```

The response generated by the local model was then displayed in the Streamlit interface.

1.5 Conversation History and Chat Management

Conversation history was implemented so that users could continue their previous conversations. The application maintains messages using Streamlit session state and also stores chat information in a JSON file.The application was enhanced with several chat-management features:

- **New Chat** – starts a fresh conversation.
- **Chat History** – allows previous conversations to be accessed.
- **Pin/Unpin** – important conversations can be kept easily accessible.
- **Rename Chat** – conversations can be given meaningful names.
- **Share/Download** – provides options for preserving or sharing conversation content.
- **Archive Chat** – older conversations can be moved away from the active list.
- **Delete Chat** – unwanted conversations can be removed.
- **Reset/New Chat functionality** – clears the current conversation and starts a new session.

The use of these features made the basic LLM inference application more similar to a practical chatbot application.

1.6 File and Image Upload

The Streamlit interface was also extended with options for interacting with uploaded files and images. These features were added to improve the usability of the chatbot interface and provide a foundation for future multimodal functionality.

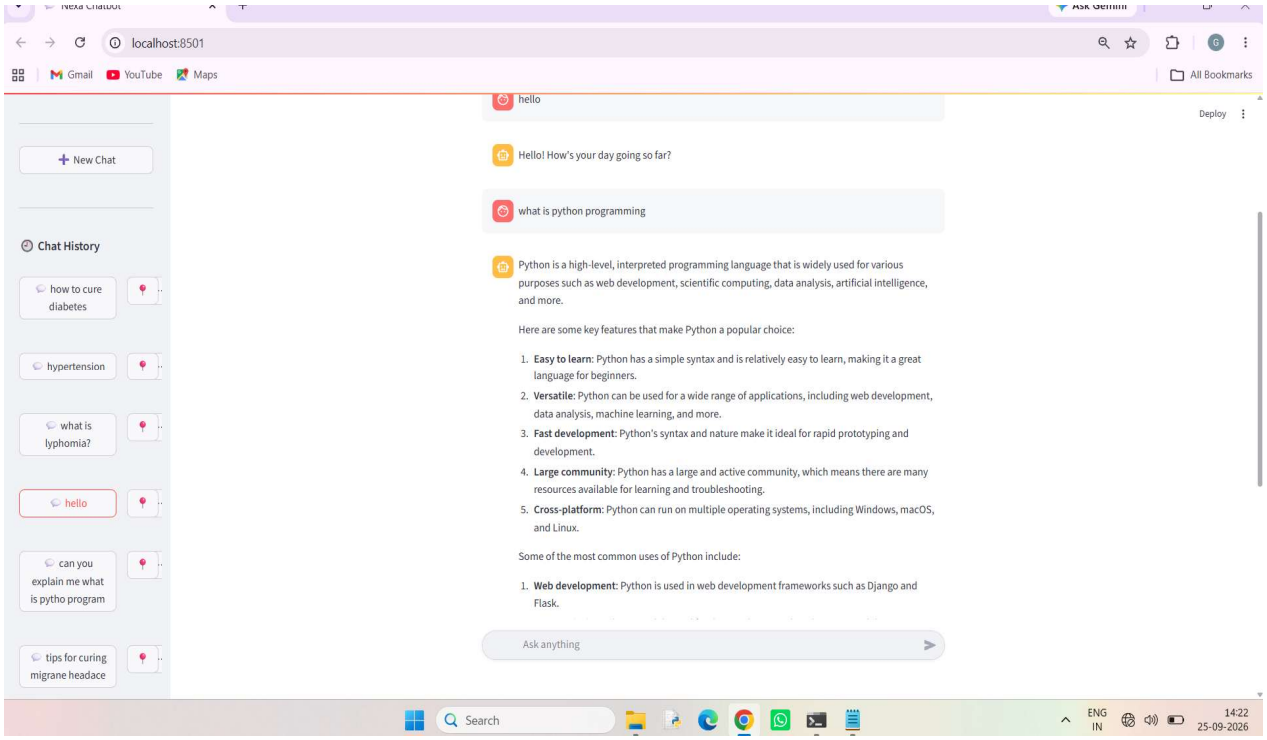


Figure 2: Nexa Chatbot Streamlit interface with chat management features

1.7 Task 1 Workflow

```
User
 ↓
Nexa Chatbot – Streamlit Interface
 ↓
User enters prompt
 ↓
Conversation stored in session state
 ↓
Ollama API / Local Endpoint
 ↓
Llama 3.2
 ↓
Generated Response
 ↓
Response displayed in Streamlit
 ↓
Conversation saved to chat history
```

Result

Task 1 successfully demonstrated local LLM inference through a Streamlit application. The application was connected to Ollama and Llama 3.2 and provided conversational interaction, history management, reset functionality, and additional chat-management features.

TASK 2 – MEDICAL FINE-TUNING WITH QUANTIZATION AND LOW-RANK ADAPTATION VIA UNSLOTH IN COLAB

2.1 Objective

The objective of Task 2 was to implement a **QLoRA-based fine-tuning workflow for a medical language model using Unsloth in Google Colab**.

The workflow included:

- Loading a domain-specific medical dataset.
- Loading a pretrained language model.
- Using 4-bit quantization.
- Configuring Low-Rank Adaptation (LoRA).
- Preparing and tokenizing the training data.
- Performing epoch-based supervised fine-tuning.
- Monitoring training loss and performance.
- Saving the trained adapter.
- Testing the fine-tuned model using new medical queries.

This directly follows the requirements shown in your internship assignment.

2.2 Google Colab and T4 GPU Setup

Task 2 was performed using **Google Colab** with an **NVIDIA Tesla T4 GPU**.

Google Colab was selected because it provides access to GPU resources required for efficient LLM fine-tuning.

Google Drive was mounted to allow the project files, dataset, training outputs, and model files to be accessed from the Colab environment.

The Drive was mounted using:

```
from google.colab import drive
```

```
drive.mount('/content/drive')
```

The project files were accessed through:

```
/content/drive/MyDrive/
```

This setup allowed the training output to be stored in Google Drive instead of depending only on temporary Colab storage.

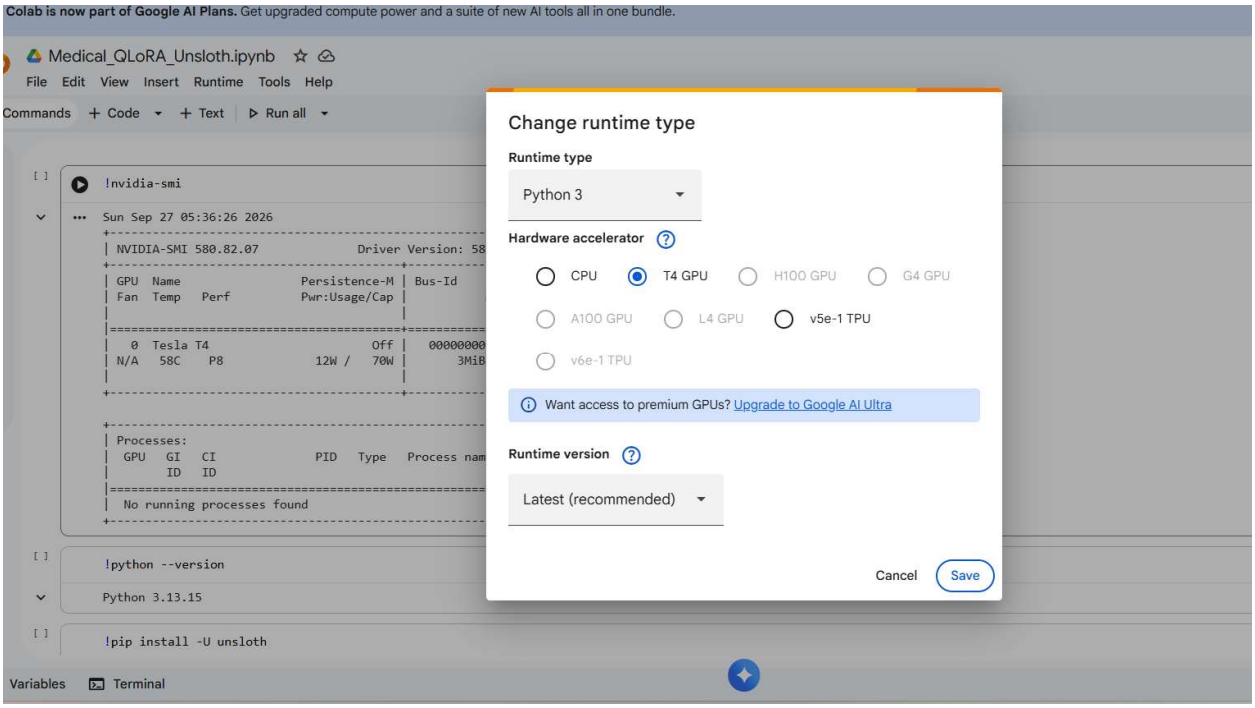


Figure 3: Google Colab environment with Tesla T4 GPU and Google Drive

2.3 Base Model

The base model used for the fine-tuning process was:

```
unsloth/Llama-3.2-1B-Instruct
```

The model was loaded using Unsloth with 4-bit loading enabled.

```
load_in_4bit=True
```

Using a smaller instruction-tuned Llama model with parameter-efficient fine-tuning made the experiment suitable for the available GPU resources.

2.4 Medical Dataset

Two datasets were experimented with during the development process.

Dataset 1 – MedQuAD

`prithvi1029/medquad-medical-qa`

This dataset follows a medical **question-answer** structure.

Dataset 2 – MedInstruct-52k

The final dataset used for the updated experiment was:

`casey-martin/MedInstruct`

The dataset was downloaded using:

`https://huggingface.co/datasets/casey-martin/MedInstruct/resolve/main/data/MedInstruct-52k.json`

MedInstruct-52k follows a medical **instruction-response** structure and was used for the final fine-tuning experiment.

ands + Code + Text ▷ Run all

Total loss records: 25

	Step	Epoch	Training Loss
0	10	0.08	1.3144
1	20	0.16	1.1682
2	30	0.24	1.0560
3	40	0.32	1.0432
4	50	0.40	1.0223
5	60	0.48	1.0395
6	70	0.56	1.0331
7	80	0.64	1.1159
8	90	0.72	1.0498
9	100	0.80	1.0174
10	110	0.88	1.0804
11	120	0.96	1.0618
12	130	1.04	0.9794
13	140	1.12	0.9375
14	150	1.20	0.9424
15	160	1.28	0.9100
16	170	1.36	0.9695
17	180	1.44	0.9106
18	190	1.52	0.9111
19	200	1.60	0.9291
20	210	1.68	0.9626
21	220	1.76	0.9493
22	230	1.84	0.9186
23	240	1.92	0.9409
24	250	2.00	0.9493

Figure 4: Loading and preparing the MedInstruct-52k medical dataset

FINE-TUNING

2.5 Dataset Preparation

The downloaded MedInstruct-52k JSON file was loaded into the Colab environment.

The dataset was prepared by organizing the medical instruction and response information into a conversational format suitable for the Llama model.

The general structure was:

```
Medical Instruction
        ↓
Medical Response
        ↓
Conversation Format
        ↓
Llama Chat Template
        ↓
Tokenization
        ↓
Training Dataset
```

The chat template converts the raw dataset into a format that resembles the actual interaction between a user and an instruction-following language model.

2.6 4-Bit Quantization

Quantization is a technique used to reduce the memory requirements of a language model by representing model parameters using lower-precision numerical formats.

In this project, **4-bit quantization** was used while loading the base model:

```
load_in_4bit=True
```

This allowed the model to be loaded using significantly less GPU memory compared with full-precision model loading.

2.7 Low-Rank Adaptation (LoRA)

Instead of updating all parameters of the pretrained model, LoRA introduces a smaller set of trainable parameters called adapter parameters.

The LoRA configuration used in the project included:

```
model = FastLanguageModel.get_peft_model(
    model,
    r=16,
    target_modules=[
        "q_proj","k_proj","v_proj","o_proj",
        "gate_proj","up_proj","down_proj",
    ],
    lora_alpha=16,
    lora_dropout=0,
    bias="none",
    use_gradient_checkpointing="unsloth",
    random_state=3407,
)
```

The selected target modules allow the adapter to modify important parts of the transformer architecture while keeping most of the original model parameters frozen.

2.8 QLoRA

QLoRA combines:

- **Quantization**
- **LoRA**
- **Parameter-Efficient Fine-Tuning**

In this project, the Llama model was loaded using 4-bit quantization and then adapted using LoRA.

The overall process was:

```
Llama 3.2 1B Instruct
          ↓
4-bit Quantization
          ↓
LoRA Adapter
          ↓
Medical Instruction Dataset
          ↓
Supervised Fine-Tuning
          ↓
Trained LoRA Adapter
```

This approach reduces the computational resources required for fine-tuning while allowing the pretrained model to learn domain-specific patterns.

2.9 Training with Unsloth

Unsloth was used to configure the parameter-efficient fine-tuning workflow.

The training process included:

- Dataset tokenization
- Chat-template formatting
- LoRA adapter configuration
- Supervised fine-tuning
- Epoch-based training
- Training-loss monitoring
- Adapter saving

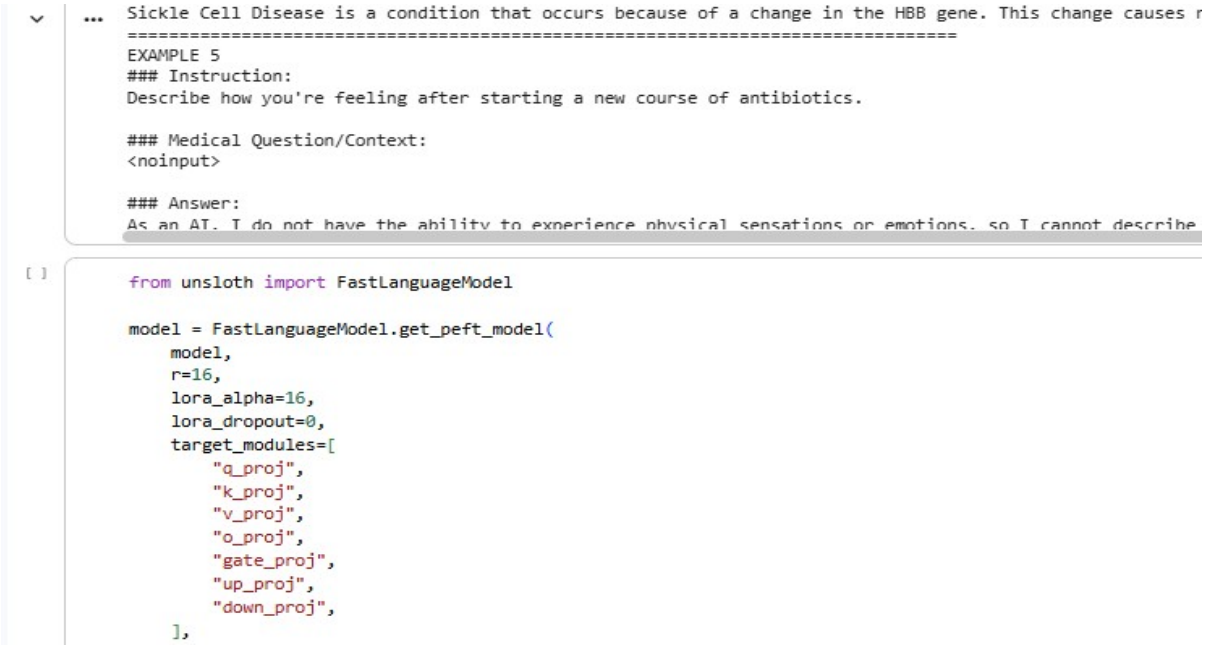


Figure 5: Unsloth and LoRA configuration for medical fine-tuning

TRAINING RESULTS

2.10 Training Monitoring

Training progress was monitored using the training-loss values reported during the training process.

The first experiment used MedQuAD, while the second experiment used MedInstruct-52k.

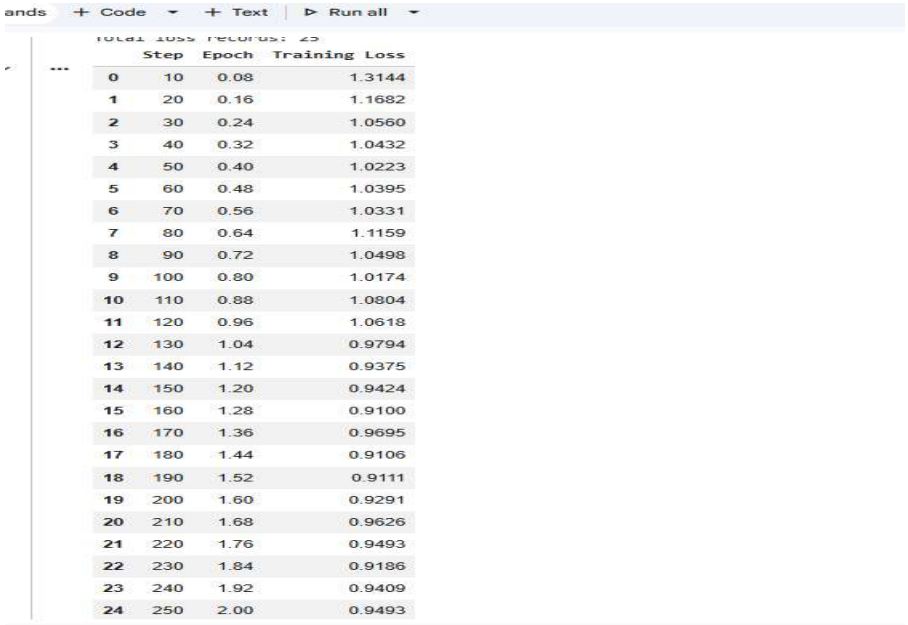
Training Loss Comparison

Dataset	Initial/Later Observed Loss	Loss at Step 250	Training
MedQuAD	2.5283 at Step 10	Approximately 1.5+ in shown output	250 steps
MedInstruct-52k	1.3144 at Step 10	**0.9493**	250 steps

For the MedInstruct experiment, the recorded values included:

```
Step 10   → 1.3144
Step 20   → 1.1682
Step 30   → 1.0560
Step 40   → 1.0432
Step 50   → 1.0223
Step 140  → 0.9375
Step 230  → 0.9186
Step 240  → 0.9409
Step 250  → 0.9493
```

The training loss generally decreased during training, indicating that the model was learning from the training examples.



	Step	Epoch	Training Loss
0	10	0.08	1.3144
1	20	0.16	1.1682
2	30	0.24	1.0560
3	40	0.32	1.0432
4	50	0.40	1.0223
5	60	0.48	1.0395
6	70	0.56	1.0331
7	80	0.64	1.1159
8	90	0.72	1.0498
9	100	0.80	1.0174
10	110	0.88	1.0804
11	120	0.96	1.0618
12	130	1.04	0.9794
13	140	1.12	0.9375
14	150	1.20	0.9424
15	160	1.28	0.9100
16	170	1.36	0.9695
17	180	1.44	0.9106
18	190	1.52	0.9111
19	200	1.60	0.9291
20	210	1.68	0.9626
21	220	1.76	0.9493
22	230	1.84	0.9186
23	240	1.92	0.9409
24	250	2.00	0.9493

Figure 6: MedInstruct-52k training loss during QLoRA fine-tuning

2.11 Saving the Fine-Tuned Adapter

After training, the LoRA adapter was saved so that the trained parameters could be reused without saving a completely new copy of the base model.

The output was stored through the Google Drive-connected project directory.

The workflow was:

```
Training
   ↓
LoRA Adapter
   ↓
Save Adapter
   ↓
Google Drive
   ↓
Reuse / Testing
```

2.13 Testing the Fine-Tuned Model

After training, the fine-tuned model/adapter was tested using new medical queries.

The purpose of testing was to verify whether the adapted model could generate responses after being fine-tuned on medical-domain instruction data.

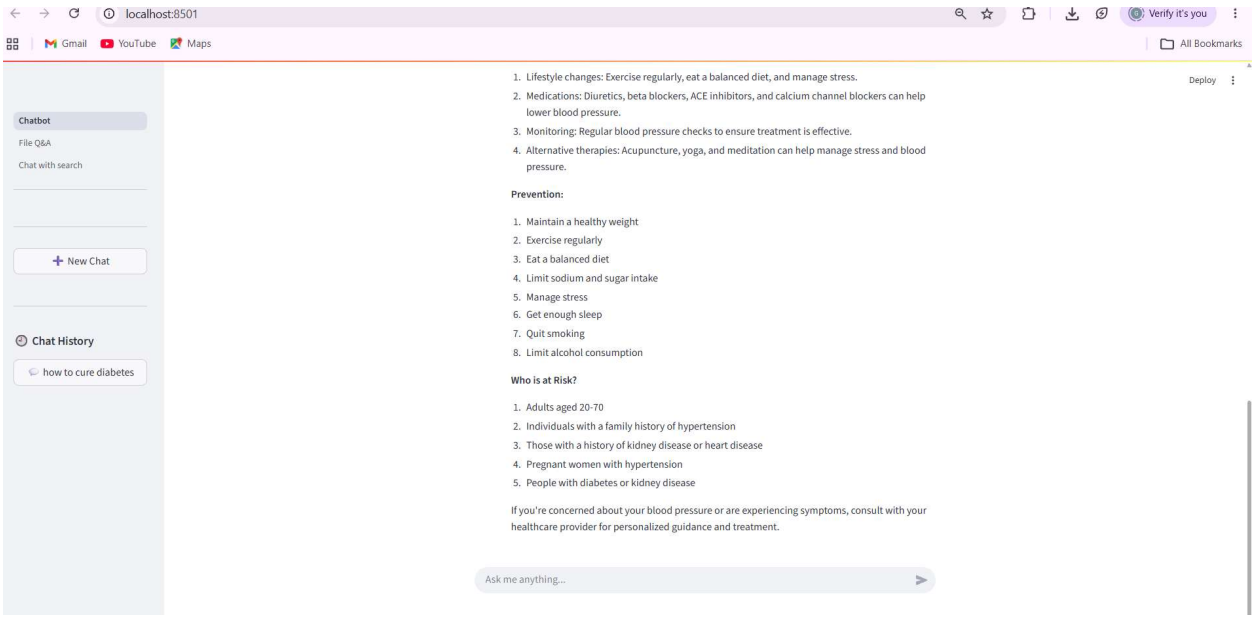


Figure 7: Testing the fine-tuned medical language model with a new medical query

INTEGRATION, LEARNING & CONCLUSION

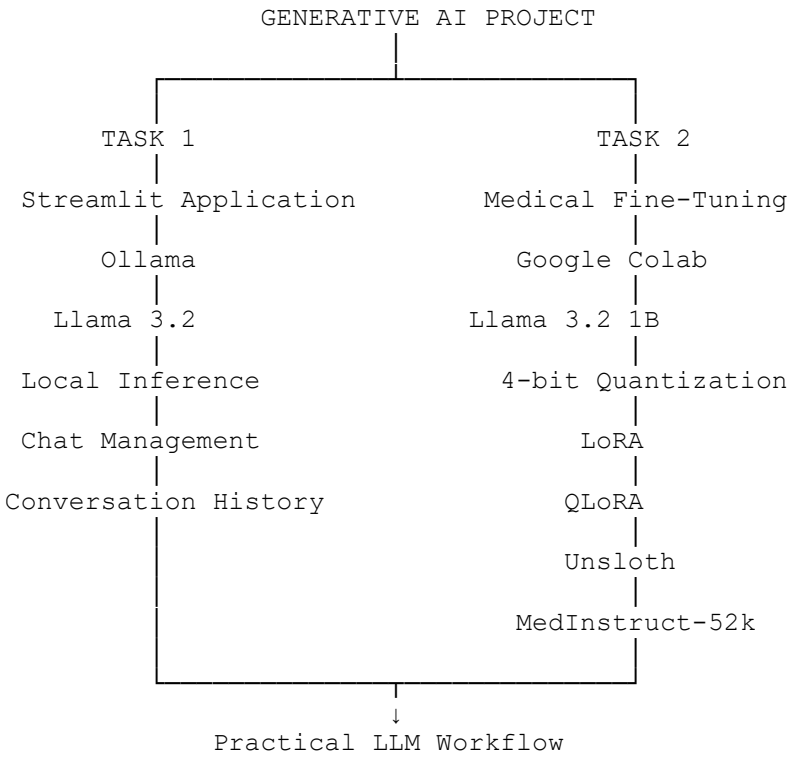
3. INTEGRATION OF TASK 1 AND TASK 2

The two tasks covered complementary parts of the Generative AI workflow.

Task 1 focused on the **application and inference side** of an LLM. A locally hosted Llama 3.2 model was connected to a Streamlit chatbot through Ollama.

Task 2 focused on the **model adaptation side**. A pretrained Llama 3.2 1B Instruct model was fine-tuned toward the medical domain using MedInstruct-52k, 4-bit quantization, LoRA, QLoRA, and Unsloth.

The overall Month 1 workflow can be represented as:



The two tasks provided practical exposure to both **using an LLM in an application** and **adapting an LLM for a specialized domain**.

4. LEARNING OUTCOMES

Through these tasks, I gained practical knowledge of:

1. Generative AI and Large Language Models.
2. Running a language model locally using Ollama.
3. Developing an interactive chatbot using Streamlit.
4. Connecting a frontend application with a locally hosted LLM.
5. Maintaining conversation history.
6. Implementing chat-management functionality.
7. Understanding Google Colab GPU environments.
8. Working with medical-domain datasets.
9. Understanding 4-bit quantization.
10. Understanding LoRA and QLoRA.
11. Using Unsloth for efficient fine-tuning.
12. Preparing instruction-response data for LLM training.
13. Monitoring training loss.
14. Saving and testing LoRA adapters.
15. Understanding the difference between LLM inference and fine-tuning.

5. CHALLENGES FACED

During the project, challenges included setting up the local Ollama environment, connecting the Streamlit application with the local model, managing persistent conversation history, preparing the medical dataset, and handling the GPU-based fine-tuning workflow.

Google Drive was used as a bridge between the local project and Google Colab. This allowed the dataset and training outputs to be accessed from the Colab environment and stored persistently. Your rough documentation specifically records mounting /content/drive/MyDrive/ and using it for project files and training outputs.

Another important learning point was understanding that **training loss should not be the only factor used to judge a dataset or model**. In the two experiments, MedInstruct produced the lower observed training loss, but the datasets have different structures and purposes.

6. CONCLUSION

The Month 1 Generative AI internship project provided practical experience in both LLM application development and model fine-tuning.

In **Task 1**, a Streamlit-based chatbot named Nexa Chatbot was developed and connected to the locally hosted Llama 3.2 model through Ollama. The application included conversational interaction, chat history, new-chat functionality, and additional chat-management features.

In **Task 2**, a medical-domain fine-tuning workflow was implemented using Google Colab, Tesla T4 GPU, Llama 3.2 1B Instruct, MedInstruct-52k, 4-bit quantization, LoRA, QLoRA, and Unsloth. The training process was monitored through training-loss values, and the trained adapter was saved for testing.

Overall, the project helped develop an understanding of the complete practical workflow of Large Language Models, from **local inference and application development to parameter-efficient domain-specific fine-tuning**.